

Etapa 4 — Resultados dos Testes e Reflexão Crítica

Autor

Marcus Vinícius Santos de Almeida

Data de entrega

04/09/2026

Cobre os entregáveis

"Resultados dos testes" (5+ casos com análise) e "Reflexão crítica"

Artefatos

testes/casos_teste.csv

testes/rodar_casos_teste.py

testes/resultados_casos.txt

Modelo de treino

150 módulos sintéticos da Etapa 2 · $P(\text{SIM}) = 52/150 = 0,347$ · $P(\text{NÃO}) = 98/150 = 0,653$

Todos os números deste relatório saem de `python3 testes/rodar_casos_teste.py`, que reconstrói o banco, roda os 6 casos e calcula os log-odds a partir da view `verossimilhanças` da Etapa 3.

1 Os 6 casos de teste

Cada caso usa as 6 features e as faixas baixo/médio/alto já definidas.

Caso	Propósito	complex.	loc	autores	churn	imports	cobertura	Intuição
a	Baixo risco claro — tudo bom	baixo	medio	baixo	baixo	baixo	alto	BAIXO
b	Alto risco claro — tudo ruim	alto	alto	alto	alto	alto	baixo	ALTO
c	Ambíguo — complexidade alta e cobertura alta	alto	medio	medio	medio	medio	alto	~50/50
d	Armadilha de Koru — pequeno e denso	alto	baixo	medio	medio	medio	medio	ALTO
e	Combinação rara — grande porém simples, churn alto	baixo	alto	baixo	alto	baixo	alto	BAIXO / indef.
f	Feature contestada — tudo ruim, cobertura alta	alto	alto	alto	alto	alto	alto	ALTO

2 Resultados

Caso	P(SIM)	P(NÃO)	Recomendação	Bate com a intuição?
a	4,17%	95,83%	BAIXO RISCO	Sim
b	95,61%	4,39%	ALTO RISCO	Sim
c	61,13%	38,87%	ALTO RISCO	Parcial — cruzou os 50%, mas é o resultado menos decidido dos 6
d	74,40%	25,60%	ALTO RISCO	Sim — o modelo capta "pequeno e denso"
e	17,12%	82,88%	BAIXO RISCO	Sim (ressalva sobre confiança — §4.3)
f	96,29%	3,71%	ALTO RISCO	Sim — cobertura alta não "salvou" o módulo

As duas probabilidades somam 100,00% em todos os casos, e as 6 features casaram com a tabela de verossimilhanças (`n_features = 6/6`).

2.1 Como cada resultado se forma

O classificador é, na prática, um somatório de log-odds: começa no log-odds a priori $\ln(52/98) = -0,634$ (a base já pende para NÃO) e soma o log-odds de cada categoria observada (tabela do §3). $P(\text{SIM}) > 50\% \Leftrightarrow \text{soma total} > 0$.

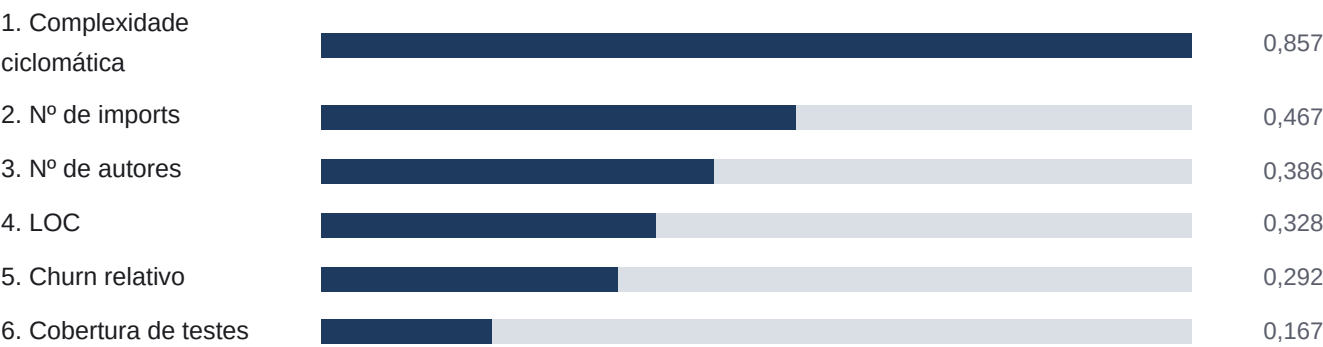
- **Caso a** — soma das features = -2,50; total -3,13 → $P(\text{SIM})$ 4%. Todas as categorias "boas" têm log-odds negativo; o modelo acumula evidência contra defeito. Correto.
- **Caso b** — soma = +3,72; total +3,08 → $P(\text{SIM})$ 96%. Espelho do caso a. Correto.
- **Caso c** — soma = +1,09; total +0,45 → $P(\text{SIM})$ 61%. A complexidade alta sozinha vale +1,41; a cobertura alta (-0,09) e o LOC médio (-0,21) apenas mordiscam essa evidência. O modelo resolve a ambiguidade a favor de ALTO RISCO, mas o placar 61/39 (contra 96/4 do caso b) mostra que os sinais contraditórios de fato se cancelaram em parte.
- **Caso d** — soma = +1,70; total +1,07 → $P(\text{SIM})$ 74%. Aqui está o ponto de Koru: LOC baixo tem log-odds +0,16 (empurra para SIM, não para NÃO) — "arquivo pequeno" não desconta risco, soma-se ao +1,41 da complexidade alta. Um arquivo LOC médio + complexidade alta teria ~0,37 de log-odds a menos.
- **Caso e** — soma = -0,94; total -1,58 → $P(\text{SIM})$ 17%. Churn alto (+0,47) e LOC alto (+0,61) puxam para SIM, mas são superados por complexidade baixa (-1,03), autores baixo (-0,46) e imports baixo (-0,45). Resultado: BAIXO RISCO. Discussão em §4.3.
- **Caso f** — soma = +3,89; total +3,26 → $P(\text{SIM})$ 96%. É o caso b com cobertura trocada de baixo para alto. O resultado *subiu* de 95,61% para 96,29%: no treino, cobertura=baixo tem log-odds -0,26 e cobertura=alto tem -0,09 — "ter cobertura alta" remove um pouco da evidência contra defeito. Aumentar a cobertura de um módulo ruim não o torna seguro aos olhos do modelo; a feature quase não pesa, e o pouco que pesa vai na direção contraintuitiva.

3 Poder discriminativo das features (log-odds)

Log-odds de uma categoria = $\ln(P(\text{categoria}|\text{SIM}) / P(\text{categoria}|\text{NAO}))$ — o "peso de evidência" daquela categoria: positivo empurra para SIM, negativo para NÃO, |valor| grande = categoria informativa.

Feature	baixo	médio	alto	Formato
complexidade	-1,03	+0,13	+1,41	monótono forte
n_imports	-0,45	-0,15	+0,80	monótono
n_autores	-0,46	-0,02	+0,68	monótono
loc	+0,16	-0,21	+0,61	em U (baixo e alto arriscados)
churn	-0,26	+0,14	+0,47	monótono fraco
cobertura	-0,26	+0,15	-0,09	plano / ruído

3.1 Ranking por |log-odds| médio (poder discriminativo)



Barras normalizadas pelo maior valor (complexidade = 0,857 → 100%). Valor exibido = |log-odds| médio das 3 categorias.

4 Análise — as 4 perguntas da atividade

4.1 (a) O modelo classificou conforme a intuição do domínio?

Em geral, sim. Os dois casos extremos (a e b) e os dois casos-armadilha (d e f) saíram exatamente como o domínio prevê.

- **Casos a e b** (baixo/alto risco claros): acertos limpos, 4% e 96% de P(SIM). O modelo empilha evidência coerente.
- **Caso c** (ambíguo): o modelo não fica em cima do muro — dá 61% e recomenda ALTO RISCO. Defensável: a complexidade alta é o sinal mais forte do modelo e a cobertura alta, o mais fraco. Mas é decisão de fronteira: se o custo de um falso alarme fosse alto, 61% talvez não bastasse para acionar revisão — o modelo entrega a probabilidade, o limiar é escolha de negócio.
- **Caso d** (Koru — pequeno e denso): acerto conceitual importante. A intuição ingênua ("arquivo pequeno → pouco código → pouco risco") levaria a BAIXO RISCO. O modelo dá 74% porque aprendeu dos dados que loc=baixo não protege (log-odds +0,16) e que a combinação com complexidade=alto acumula risco — reproduz o achado de Koru et al. (2008).
- **Caso e** (raro): resultado plausível (BAIXO RISCO, 17%), com a ressalva de §4.3 sobre o modelo soar mais confiante do que deveria para um perfil que nunca viu.
- **Caso f** (cobertura contestada): acerto — pelo motivo certo. Cobertura alta não derrubou o risco de um módulo ruim em tudo o mais (P(SIM) 96%). O modelo não "confia demais" na cobertura simplesmente porque o treino não deu peso a ela (log-odds ≈ 0) — comportamento desejável para uma feature de evidência contestada.

4.2 (b) Quais features tiveram maior log-odds?

1. **Complexidade ciclomática** domina, com folga ($|\log\text{-odds}|$ médio 0,857; a categoria alto sozinha vale +1,41 — $P(\text{alto}|\text{SIM})$ é $\sim 4 \times P(\text{alto}|\text{NAO})$).
2. **Nº de imports e nº de autores** formam um segundo grupo ($\sim 0,4$): a categoria alto de ambas é claramente informativa (+0,80 e +0,68).
3. **LOC** é o caso mais interessante: log-odds em U — baixo (+0,16) e alto (+0,61) empurram para SIM, médio (-0,21) para NÃO. O modelo capturou a não-linearidade de Koru (§4.1, caso d).
4. **Churn relativo** é fraco ($\sim 0,29$): só a categoria alto (+0,47) diz algo.
5. **Cobertura de testes** é a menos discriminativa de todas (0,167; máx. 0,26). Suas três categorias têm log-odds perto de zero e não são monótonas — consistente com a literatura que trata cobertura $\leftrightarrow$ defeitos como contestada (Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017), e com o modo como a Etapa 2 gerou a feature (efeito propositalmente fraco).

4.3 (c) Valor não visto / na fronteira (caso e) e o papel de Laplace

O perfil do caso e (complexidade baixo, loc alto, n_autores baixo, churn alto, n_imports baixo, cobertura alto) não aparece em nenhum dos 150 módulos de treino (perfil exato no treino = 0). Ainda assim o classificador devolve uma resposta bem definida: $P(\text{NÃO}) = 82,9\%$.

Por que ele consegue responder

O Naive Bayes nunca precisa ter visto o perfil completo — pela hipótese de independência, ele só usa as seis fatias unidimensionais $P(\text{categoria}|\text{classe})$, e cada uma está bem povoada. A esparsidade da combinação de 6 categorias (729 perfis possíveis, 150 exemplos) não trava o modelo. Essa é a força do Naive Bayes — e também o seu risco: ele soa confiante (83/17) sobre uma combinação contraditória ("arquivo grande, muito alterado, mas simples e pouco acoplado") que talvez mereça mais incerteza.

O que a suavização de Laplace faz aqui

Neste dataset, nenhuma célula (feature, categoria, classe) está vazia — a menor contagem é 6 (churn=alto | SIM). Laplace não precisou impedir nenhum zero neste teste; o que faz é encolher levemente cada estimativa na direção do uniforme: $P(\text{churn=alto}|\text{SIM})$ passa de $6/52 = 0,115$ (sem suavização) para $(6+1)/(52+3) = 0,127$. Efeito pequeno porque as células têm dados suficientes.

Quando Laplace seria decisivo: com mais faixas ou menos dados, alguma célula ficaria em 0 e — sem suavização — $\ln(0) = -\infty$ zeraria a classe inteira por causa de uma categoria não observada. Com Laplace, essa célula valeria $1/55 \approx 0,018$ em vez de 0, e a classe continuaria "viva" para as outras 5 features decidirem. É uma apólice de seguro que, neste conjunto, não precisou ser acionada — mas mantém o modelo bem-definido para qualquer entrada.

4.4 (d) Limitações do Naive Bayes neste domínio

1. Violação de independência: complexidade × LOC

É a limitação mais séria. As duas features têm correlação empírica altíssima ($R^2 \approx 0,93$ na literatura; 0,67 de Pearson nos nossos dados sintéticos, ainda a maior da matriz). O Naive Bayes assume que são evidências independentes e soma os dois log-odds como se fossem informações separadas — na prática, conta o "tamanho/estrutura do código" quase duas vezes. Efeito: o modelo fica mais confiante do que deveria sempre que complexidade e LOC apontam na mesma direção (boa parte dos casos b, d, f). Um modelo que modelasse a correlação (regressão logística, ou fundir as duas numa só feature) daria probabilidades menos extremas.

2. Relação não-linear de LOC com risco. LOC não é "quanto maior, pior": pela lei de potência de Koru et al., arquivos pequenos são proporcionalmente mais arriscados. A discretização em 3 faixas contorna isso (o log-odds em U do §3 mostra que funcionou), mas o Naive Bayes só representa esse padrão porque loc=baixo e loc=alto acabaram ambos com log-odds positivo — ele não modela a interação "loc baixo E complexidade alta" diretamente. No caso d, o efeito "pequeno e denso" aparece só como a soma de dois termos individuais; se a interação real fosse maior que a soma das partes, o modelo a subestimaria.

3. Feature de evidência contestada: cobertura de testes. Incluída sabendo que sua relação com defeitos é fraca/disputada na literatura. O ranking confirmou: é a feature menos discriminativa, com log-odds ≈ 0 e não monótono. Praticamente não ajuda e ainda adiciona ruído (caso f: a cobertura alta empurrou o score na direção "errada"). Mantê-la é limitação assumida, não descuido — mas é peso morto no modelo.

4. Simplificação módulo = arquivo (não classe). O estudo de referência (Koru et al.) usa classe como unidade; adotamos arquivo por ser universal e não exigir parser por linguagem. Um arquivo agrega várias classes/funções, então as métricas por registro ficam mais "grossas" e um sinal de defeito concentrado numa classe específica se dilui na média do arquivo.

5. Dados de treino sintéticos. Os 150 módulos foram gerados por script (Etapa 2), com padrões programados a partir da literatura. O modelo aprende exatamente os padrões que embutimos — a validação contra a "intuição do domínio" é, em parte, circular. Com dados reais os log-odds seriam mais ruidosos e a cobertura poderia até inverter de sinal.

5 Reflexão crítica

O classificador acerta o que se pede dele num cenário de priorização: separa com folga os módulos claramente arriscados dos claramente seguros (casos a e b, 4% vs. 96%) e — mais importante — acerta os dois casos em que a intuição ingênua erraria. No caso d ele trata um arquivo pequeno e denso como alto risco, porque aprendeu dos dados que tamanho pequeno não é sinônimo de segurança (o achado contraintuitivo de Koru et al.); no caso f ele não deixa uma cobertura de testes alta "limpar a barra" de um módulo ruim em todo o resto, porque o treino simplesmente não deu peso a essa feature. Nos dois casos o comportamento correto emerge dos dados, não de uma regra escrita à mão — e é isso que um classificador bayesiano bem alimentado deveria fazer.

Onde ele falha é onde a hipótese "naive" cobra o preço. Como complexidade ciclomática e LOC carregam quase a mesma informação ($R^2 \approx 0,93$ na literatura) mas entram como duas evidências independentes, o modelo conta o tamanho do código duas vezes e sai mais confiante do que os dados justificam sempre que essas duas features concordam — o caso na maioria dos perfis de alto risco. Pela mesma razão, ele não representa interações: o risco extra de "pequeno e denso" aparece só como a soma de dois termos separados, nunca como um efeito próprio da combinação. Some-se a isso que uma das seis features (cobertura) é quase informação nula e que o rótulo de treino é sintético — construído a partir dos mesmos padrões contra os quais estamos "validando".

Veredito honesto

O modelo é uma boa ferramenta de **triagem** — rápido, transparente, explicável linha a linha, e suas probabilidades ordenam bem os módulos do mais para o menos suspeito — mas os números absolutos de confiança (95%, 96%) devem ser lidos como "muito provável", não como probabilidades calibradas. Para uso real, o passo seguinte seria reduzir a redundância complexidade/LOC (fundir as duas ou trocar por um modelo que aceite correlação) e recalibrar tudo contra defeitos observados num repositório de verdade.

Nota sobre o uso de IA

Os casos de teste, o script de execução e esta análise foram construídos em diálogo com uma IA generativa. Cada resultado numérico foi reproduzido pelo script (`testes/rodar_casos_teste.py`) e conferido contra a decomposição em log-odds; as limitações teóricas foram checadas contra as referências levantadas na Etapa 1. Todo o conteúdo é defensável oralmente.